

An Energy-Efficient TDMA Algorithm for Energy Harvesting Wireless Sensor Networks

Robert J. Hayek  and Ahmed Ammar 

Department of Electrical Engineering and Computer Science

Ohio Northern University, Ada, Ohio, USA

r-hayek@onu.edu and a-ammar@onu.edu

Abstract—Recently, it has been shown that energy harvesting can be utilized to enhance the energy efficiency of wireless sensor networks. However, energy harvesting is a stochastic process, which in turn imposes an energy availability constraint on the system’s design, necessitating the demand for communication and networking protocols and algorithms that can handle this constraint. In this paper, we propose a time division multiple access (TDMA) system and algorithm for a single-hop energy harvesting wireless sensor network. Unlike the existing algorithms, the proposed algorithm synchronizes nodes not only based on the synchronization packets from the central node but also based on a data packet from another node. In addition, the proposed algorithm synchronizes nodes only when all of them are out-of-energy or when a transmission overlap occurs. We implemented the proposed algorithm on an Arduino-based network. Experimental results show that the proposed algorithm significantly reduces the average energy consumption and the average delay while handling clock drift between nodes. Compared to the most recent algorithm, the proposed algorithm reduces the average energy consumption by approximately 7 times and decreases the average delay by approximately 12 times.

Index Terms—Time Division Multiple Access, Wireless Sensor Networks, Energy Harvesting, Synchronization, Clock Drift.

I. INTRODUCTION

Wireless sensor networks (WSNs) have a wide range of applications [1], where a set of sensor nodes are deployed to collect, process, and transmit data to a central unit. The size of sensor nodes is typically small, i.e., a few cubic millimeters [2], which limits their energy source, resulting in significant lifetime limitations and causing high maintenance costs due to battery replacement or recharging. Many energy-efficient protocols have been proposed to reduce the severity of these issues [3]–[5]; however, energy efficiency alone does not eliminate them. Hence, energy harvesting has been recently proposed as an alternative solution to drastically improve lifetime, and handle battery recharging to reduce costs.

Although it drastically improves the performance of WSNs, energy harvesting places more constraints on the system’s design due to the stochastic nature of the energy harvesting processes. Thus, all the conventional algorithms of WSNs, e.g., scheduling, synchronizing, communication, routing, and tracking algorithms need to be modified to accommodate the new constraints. Recently, some existing algorithms have been revisited and new algorithms have been proposed [6]–[13].

The authors in [11]–[13] propose TDMA algorithms for a single-hop energy harvesting wireless sensor network, which is

similar to our work in this paper. The TDMA algorithms CEH-TDMA and AT-MAC proposed in [11] and [12], respectively, synchronize the network only once, and allocate time slots for the nodes periodically based on the information received from each node in each period. Synchronizing a network only once is impractical because of the clock drift between sensor nodes [14], [15]. Clock drift causes nodes to be out of synchronization after some time from the initial synchronization, resulting in transmission overlaps and network malfunction. The TDMA algorithm EH-TDMA proposed in [13] eliminates the clock drift effect by synchronizing the network periodically. However, the synchronization interval is short, i.e., every time the time slots are allocated, which results in a high number of synchronization packets and thus high energy consumption and delay.

In this paper, we propose a TDMA algorithm that synchronizes the network less often than the EH-TDMA algorithm while handling the clock drift problem. The proposed algorithm decreases the synchronization events by increasing the synchronization interval, which is increased by 1) allowing the sensor nodes to synchronize not only based on the synchronization packets from the central node as in [11]–[13], but also based on a data packet from another node, and 2) synchronizing the sensor nodes only when all nodes are out-of-energy or when a transmission overlap occurs, which occurs due to clock drift.

We implemented the proposed algorithm on an Arduino-based wireless sensor network. We evaluated the performance of the proposed algorithm based on the average energy consumption and the average delay per synchronization interval. Average energy consumption is defined as the average energy consumed by a node plus the average energy consumed by the central node due to synchronization and time slot allocation. Average delay is defined as the total time allocated for synchronization and time slot allocation. Compared to EH-TDMA, experimental results show that the proposed algorithm reduces the average energy consumption by approximately 7 times and decreases the average delay by approximately 12 times. The performance achieved by CEH-TDMA and AT-MAC algorithms is the worst, which is expected because the two algorithms were primarily designed to maximize channel utilization and the amount of packet transmission.

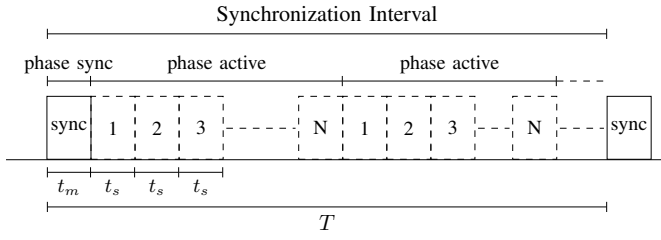


Fig. 1. Structure of the proposed TDMA system.

II. SYSTEM MODEL

We consider a single-hop wireless network that consists of a central node and N wireless sensor nodes. As in [13], the central node is called *coordinator* and coordinates the network synchronization, transmissions, and data aggregation. We assume that each sensor node has a pre-assigned unique identification number, i.e., a number from 1 to N . We denote the identification number of sensor node i , $i \in \{1, 2, \dots, N\}$, by ID_i , where $ID_i = i$. Each node transmits to the coordinator in a different time slot and includes its identification number in its transmission packet. As in [13], we assume that nodes always have data to transmit in each time slot.

Similar to [12], we assume that only sensor nodes are energy harvesting. The nodes follow the *Harvest-Store-Spend* policy as in [6] and [16], where the energy harvested in a time slot is stored in the battery and used in a later time slot. We assume that the energy harvesting processes of the nodes are independent. Thus, similar to [6] and [16], we model the event that node i , $i \in \{1, 2, \dots, N\}$, has sufficient energy in a time slot as a Bernoulli random variable with probability p . Thus, the event that node i is out-of-energy will also be a Bernoulli random variable, but with probability $1 - p$. The out-of-energy event of node i is denoted by O_i , where $O_i \in \{0, 1\}$.

III. PROPOSED TDMA SYSTEM

The proposed TDMA system, as shown in Fig. 1, is comprised of two phases: synchronization (*SYNC*) phase and active (*ACTIVE*) phase. Phase *SYNC* forces a measure of synchronization of the nodes. Phase *ACTIVE* immediately follows phase *SYNC*, during which nodes transmit their data packets consecutively.

Phase *ACTIVE* is divided into N time slots each of length t_s . Each time slot is assigned to one node such that time slot i , $i \in \{1, 2, \dots, N\}$, is assigned to node i . This phase will repeat as long as no error is detected. An error is defined as either a transmission overlap between the nodes' transmissions, which occurs due to clock drift between sensor nodes, or all nodes are out-of-energy. If an error occurs, it means that nodes have become out-of-sync. Thus, phase *SYNC* will start again, forcing the nodes to resynchronize, and then phase *ACTIVE* will start, repeating the same process.

As illustrated in Fig. 1, the time interval between two consecutive synchronization phases is the synchronization interval, which is denoted by T and can be expressed as $T = t_m + MNt_s$, where t_m is the time length of phase *SYNC*

Algorithm 1: Coordinator Algorithm

```

1 begin
2   set phase to SYNC;
3   switch phase do
4     case SYNC do
5       broadcast sync packet;
6       set phase to ACTIVE;
7     end
8     case ACTIVE do
9       while (no errors) do
10        receive packets and check for errors;
11      end
12      if out-of-energy error then
13        set phase to SYNC;
14      end
15      if transmission overlap error then
16        halt the network by broadcasting a sync
17        packet;
18        wait for an acknowledgment ;
19        if acknowledgement received then
20          set phase to SYNC;
21        end
22      end
23    end
24  end

```

and M is the number of active phases per synchronization interval. The number of active phases M is random because errors occur randomly due to the randomness of clock drifts and energy availability at nodes, resulting in a random synchronization interval T . Therefore, the proposed algorithm is evaluated based on the average performance, i.e., the average energy consumption and the average delay. These two metrics and the average number of active phases $\bar{M}$ are calculated per the average synchronization interval $\bar{T}$. Although T is random, the proposed algorithm still synchronizes the nodes less often than the EH-TDMA algorithm, as we will see later in Section VI.

IV. PROPOSED ALGORITHM

This section describes the proposed algorithm that implements the TDMA system proposed in Section III. The proposed algorithm is comprised of two algorithms: Algorithm 1 and Algorithm 2, which are described next.

A. Algorithm 1

Algorithm 1 is run at the coordinator. As can be seen from the algorithm, the coordinator has two operating phases: *SYNC* and *ACTIVE*. Upon booting up, the coordinator starts at phase *SYNC*, where it first broadcasts a *sync* packet and then immediately transitions to phase *ACTIVE*. In phase *ACTIVE*, the coordinator receives data packets from sensor nodes and checks for errors. The coordinator remains in phase *ACTIVE*

Algorithm 2: Sensor Node Algorithm

```

1 begin
2   set phase to DEAD;
3   switch phase do
4     case DEAD do
5       harvest energy;
6       if sufficient energy collected then
7         set phase to SYNC;
8       end
9     end
10    case SYNC do
11      wait for a synchronization packet;
12      if a synchronization packet received then
13        set phase to ACTIVE;
14      end
15    end
16    case ACTIVE do
17      synchronize using the synchronization
        packet;
18      while (still has sufficient energy) do
19        wait for the designated time slot;
20        send packet;
21        if sync packet received then
22          send acknowledgment ;
23          set phase to SYNC;
24        end
25      end
26      set phase to DEAD;
27    end
28  end
29 end

```

as long as no error is detected. If an error is detected, meaning nodes have become out-of-sync, the coordinator will transition back to phase *SYNC*, calling the nodes for resynchronization. The coordinator transitions back to phase *SYNC* either immediately or after receiving an acknowledgment, depending on the type of error. If the detected error is out-of-energy, the coordinator will transition immediately to phase *SYNC*. If the detected error is transmission overlap, the coordinator will first halt the network by broadcasting a *sync* packet and then wait for an acknowledgment from nodes. If at least one acknowledgment is received, the coordinator will transition to phase *SYNC*. After transitioning to phase *SYNC*, the coordinator will repeat the same process. Section V-A discusses how the coordinator checks and detects errors.

B. Algorithm 2

Algorithm 2 is run at sensor nodes. As can be seen from the algorithm, sensor nodes have an additional operating phase compared to the coordinator, and this phase is *DEAD*. When a sensor node boots, it starts at phase *DEAD* and remains in this phase until it has sufficient energy. When the node has sufficient energy, it transitions to phase *SYNC* in which it waits

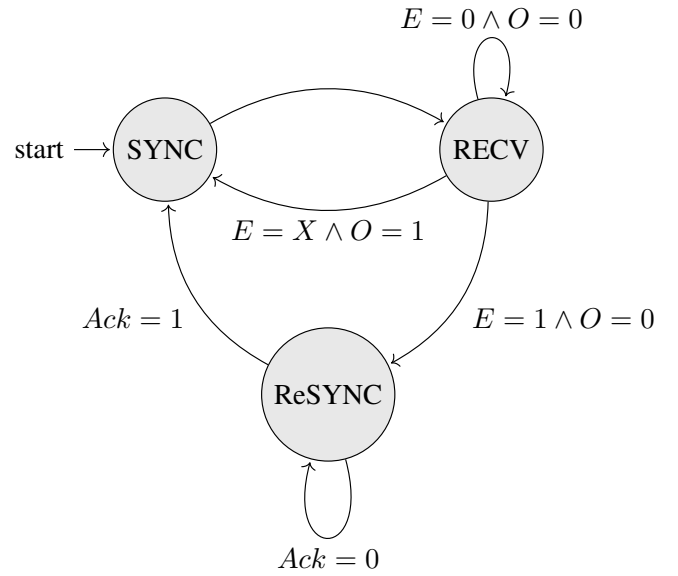


Fig. 2. State Diagram of Algorithm 1.

for a synchronization packet to synchronize. A synchronization packet is either a *sync* packet from the coordinator or a data packet from another node. If a synchronization packet is received, the node will immediately transition to phase *ACTIVE*, where it first synchronizes using the synchronization packet and then waits for its designated time slot to transmit its data packet. Section V-B discusses how a node synchronizes and waits for its designated time slot. The node remains in phase *ACTIVE* as long as it has sufficient energy and no *sync* packet is received from the coordinator. If the node runs out of energy, it will transition to phase *DEAD*, while if it receives a *sync* packet, it will transition to phase *SYNC*. From these two phases, the node will continue the same process.

V. ALGORITHM IMPLEMENTATION

We implemented the proposed algorithms as Finite State Machines (FSMs) for simple, fast, and easy troubleshooting and coding. The FSMs are described next.

A. FSM for Algorithm 1

The FSM of Algorithm 1 is shown in Fig. 2. As illustrated in the figure, the FSM has three states: synchronization state (*SYNC*), receive state (*RECV*), and resynchronization state (*ReSYNC*). State *SYNC* implements phase *SYNC* in Algorithm 1 while states *RECV* and *ReSYNC* implement phase *ACTIVE*.

Upon starting up, as illustrated by Fig. 2, the coordinator node enters state *SYNC*, where it first broadcasts a *sync* packet and then immediately transitions to state *RECV*. In state *RECV*, the coordinator receives packets from the nodes and also uses the received packets to detect out-of-energy and transmission overlap errors, which are denoted by *O* and *E*, respectively. The coordinator remains in state *RECV* as long as no error is detected, i.e., $O = 0$ and $E = 0$. If an out-of-energy error is detected, i.e., $O = 1$, the coordinator will transition

to state *SYNC* regardless of E to start the process again. If there is no out-of-energy error and a transmission overlap error is detected, i.e., $E = 1$ and $O = 0$, the coordinator will transition to state *ReSYNC*. In state *ReSYNC*, the coordinator broadcasts a *sync* packet and then waits for an acknowledgment from at least one node, i.e., $Ack_i, i \in \{1, 2, \dots, N\}$. If at least one acknowledgment is received, i.e., $Ack = 1$, where $Ack = \bigcup_{i=1}^N Ack_i$, the coordinator will transition to state *SYNC* to start the process again.

As mentioned earlier, the coordinator in state *RECV* uses the received packets to detect errors. The coordinator detects transmission overlap errors by making sure that data packets are received in their designated time slots, i.e., a data packet of node i is received in time slot i , and also by making sure the time duration between any two consecutive received packets is at least t_s . To check whether packets are received in their designated time slots or not, the coordinator starts a counter b as soon as it transitions to state *RECV*, and increments this counter every time a data packet is received. For each received data packet, the coordinator extracts the identification number contained in that packet, say ID_b . The coordinator then checks the logic statement: $b \bmod N = ID_b \bmod N$. If this statement is not true, it means that the current packet is not received in the correct time slot. On the other hand, to check the time duration between two consecutive received packets, the coordinator starts recording the timestamp of the received packets as soon as it transitions to state *RECV*, and at the beginning of the receiving process of the packets. The coordinator only keeps the timestamp of the current and the previous received packets t_c and t_p , respectively. After receiving a packet, the coordinator calculates the time difference $t_{diff} = t_c - t_p$, which should be at least t_s . If $|t_{diff} - t_s| \leq \varepsilon$, where ε is a small tolerance value, it means an overlap has occurred between the two packets. Hence, if $|t_{diff} - t_s| \leq \varepsilon$ or the packets are not received in the correct order, the coordinator will declare a transmission overlap error, i.e., $E = 1$. As for the out-of-energy error, the coordinator starts a timer of $Nt_s \pm \varepsilon$ seconds after each received packet. If the timer is done and no packet is received, the coordinator will declare an out-of-energy error, i.e., $O = 1$.

B. FSM for Algorithm 2

The FSM of Algorithm 2, as shown in Fig. 3, has four states: *DEAD*, *SYNC*, *WAIT*, and *TX*. State *DEAD* implements phase *DEAD* in Algorithm 2, state *SYNC* implements phase *SYNC*, and states *WAIT* and *TX* implement phase *ACTIVE*.

Upon starting up, as can be seen from Fig. 3, node i starts at state *DEAD* and enters this state whenever it becomes out-of-energy, i.e., $O_i = 1$. When node i collects sufficient energy, i.e., $O_i = 0$, it transitions immediately to state *SYNC*. In this state, node i waits for a synchronization packet. If a synchronization packet is received, i.e., $s_i = 1$, node i transitions to state *WAIT* where it waits for its assigned time slot to transmit its data packet. When its time slot is up, i.e., $\tau_i = 1$, node i transitions to state *TX*, where in this state, node i transmits its packet and then transitions to state *WAIT* if

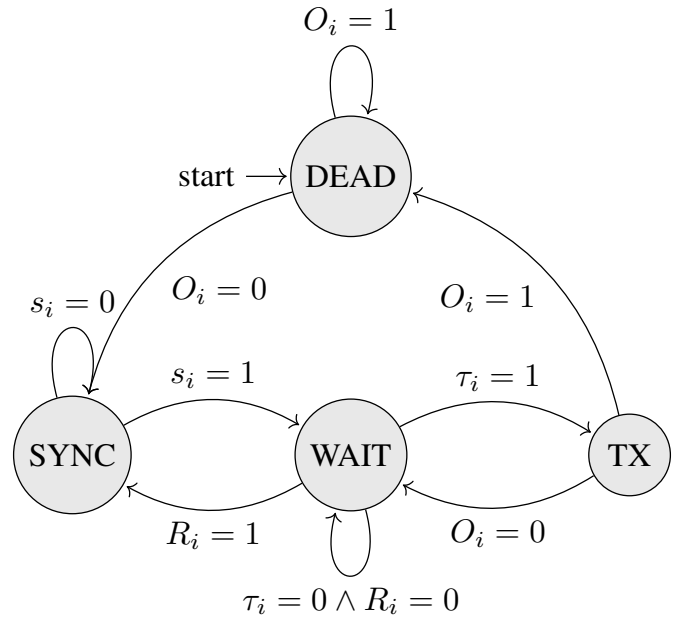


Fig. 3. State Diagram of Algorithm 2.

sufficient energy remained or state *DEAD* if otherwise. While in state *WAIT*, if $R_i = 1$, i.e., a *sync* packet is received from the coordinator due to a transmission overlap, node i will transition to state *SYNC* for resynchronization, and repeat the same process.

As mentioned earlier, in state *WAIT*, node i waits for its assigned time slot to transmit its data packet. The node uses the synchronization packet received in phase *SYNC* to wait for its designated time slot as follows. Node i starts a counter c_i whenever it enters state *WAIT*. The counter increments every t_s seconds, and to ensure that the current value of c_i indicates the current time slot, the initial value of c_i is set based on the received synchronization packet. If the synchronization packet is a *sync* packet from the coordinator, then the initial value of c_i will be set to 1. If the synchronization packet is a data packet from another node, say node j , then the initial value of c_i will be set to ID_j , which, as defined in Section II, is the identification number of node j and is assumed to be included in the transmitted data packet. Whenever counter c_i is incremented, node i will check the logic statement: $c_i \bmod N = i \bmod N$. If this is true, node i will set τ_i to 1, which means that the current time slot is the time slot of node i . Otherwise, node i will set τ_i to 0 and keep waiting.

VI. EXPERIMENTAL RESULTS

This section describes the experimental setup and some results obtained by implementing the proposed algorithm on an Arduino-based wireless sensor network. Additionally, our source code is available on GitHub.

A. Network Setup

Similar to [13], the experiments have been conducted on a network of four nodes; three sensor nodes and a coordinator.

Each node is comprised of an Arduino Uno R3 and a Digi XBee S2C. The Digi XBee modules were mounted on the Arduino boards using Arduino Xbee shields.

A separate XBee module was used to collect data from the network for performance analysis. This XBee module was connected to a serial port on a PC. A Python script was written to time, count, and read all the data packets received by this XBee module and to save this data to a text file. Data is stored in the text file with a comma-delimited format to allow simple importing for analysis. Each entry in the text file represents one received packet and contains the reception timestamp, packet number, and data in the packet. The stored data were used to calculate the average synchronization interval $\bar{T}$ and the average number of active phases $\bar{M}$.

B. Verification

We first run the proposed algorithm for $p = 1$ and for different lengths of the time slot t_s . $p = 1$ means that the nodes will always have sufficient energy and all the resynchronization events will occur only due to the overlap transmission error. For each value of t_s , we ran 5 trials and each trial ran for at least two hours. We used the collected data to calculate the average synchronization interval per trial, and then we averaged over the 5 trials to get the average synchronization interval $\bar{T}$. Table I shows the average synchronization interval $\bar{T}$ per each value of t_s . As expected, $\bar{T}$ increases as t_s increases. Note that 56 ms was found experimentally to be the minimum length for t_s .

TABLE I
AVERAGE SYNCHRONIZATION INTERVAL $\bar{T}$ FOR DIFFERENT t_s VALUES

$\bar{T}$ /trial (s)	Time Slot Length t_s (ms)						
	56	500	1000	1500	2000	2500	3000
Trial 1	5.78	572.32	681.04	1755.73	1359.75	2982.17	3559.12
Trial 2	2.40	567.26	677.84	1014.11	1395.09	2972.18	3581.83
Trial 3	2.18	574.28	677.23	1011.41	1387.57	2970.81	3577.11
Trial 4	2.45	577.87	679.67	1022.68	1386.14	2971.13	3587.93
Trial 5	2.53	572.95	681.88	1025.74	1387.10	2953.23	3563.93
$\bar{T}$	3.07	572.93	679.53	1165.93	1383.13	2969.90	3573.98

We then repeated the same experiment for $p = 0.8$ and $p = 0.4$. Here, for each t_s , we calculated the average number of active phases per trial, and then we averaged over the 5 trials to get the average number of active phases $\bar{M}$. Tables II and III show the average number of active phases $\bar{M}$ for each value of t_s . After analyzing the collected data, we found that all the resynchronization events occurred due to the out-of-energy error. None of the resynchronization events was due to the transmission overlap. This is because when a node becomes out-of-energy, it resynchronizes based on another node, which negates the clock drift between the nodes.

As can be seen from Tables II and III, the average number of active phases $\bar{M}$ is 123.99 and 4.55 when $p = 0.8$ and $p = 0.4$, respectively. These experimental values are as expected because independent energy harvesting processes result in independent out-of-energy events of the nodes. Thus, the probability that all nodes are out-of-energy in an active phase is $(1 - p)^N$, where $N = 3$ because there are only three

TABLE II
AVERAGE NUMBER OF ACTIVE PHASES $\bar{M}$ WHEN $p = 0.8$

$\bar{T}$ /trial	Time Slot Length t_s (ms)				
	56	500	1000	1500	2000
Trial 1	115.14	129.09	132.32	120.07	126.79
Trial 2	116.26	114.14	134.60	138.09	145.42
Trial 3	109.35	129.48	128.89	132.76	125.42
Trial 4	112.35	120.32	122.03	160.73	113.29
Trial 5	111.67	124.75	100.83	144.27	91.82
Average	112.95	123.56	123.73	139.18	120.55
$\bar{M}$	123.99				

TABLE III
AVERAGE NUMBER OF ACTIVE PHASES $\bar{M}$ WHEN $p = 0.4$

$\bar{T}$ /trial	Time Slot Length t_s (ms)				
	56	500	1000	1500	2000
Trial 1	4.82	4.55	4.64	4.55	4.69
Trial 2	4.65	4.65	4.69	4.50	4.78
Trial 3	4.54	4.70	4.71	2.72	4.28
Trial 4	4.52	4.58	4.58	4.57	4.77
Trial 5	4.73	4.74	4.64	4.56	4.59
Average	4.65	4.64	4.65	4.18	4.62
$\bar{M}$	4.55				

nodes in the network. Consequently, the event that the active phase continues until all the nodes become out-of-energy can be modeled as a Bernoulli trial. Hence, the average number of active phases before all nodes run out of energy should be $1/(1 - p)^3$. When $p = 0.8$ and when $p = 0.4$, the average number of active phases is 125 and 4.63, respectively, and these values are approximately the same as the experimental values shown in Tables II and III.

C. Performance Evaluation

The performance of the proposed algorithm is compared with the performance of AT-MAC [12] and EH-TDMA [13]. CEH-TDMA is not included in the comparison as it achieves the same performance as AT-MAC. As stated in Section II, the performance is defined in terms of the average energy consumption and the average delay. For fair comparison, we compared the algorithms when $p = 1$, and we set the time length of the ping message in the EH-TDMA algorithm and the time length of the information packet, the time length of the schedule packet, and the time length of the beacon packet in the AT-MAC algorithm equal to the length of the *sync* packet in our algorithm. We also set the transmit power and receive power to 1W since different hardware was considered.

1) *Average energy consumption of a sensor node:* For the proposed algorithm, the energy consumption of a sensor node per $\bar{T}$ is equal to the energy consumption of receiving a *sync* packet plus the energy consumption of transmitting an acknowledgment. As for the EH-TDMA algorithm, the energy consumption of a sensor node per $\bar{T}$ is equal to the energy consumption of receiving $\bar{T}/(t_m + Nt_s)$ *sync* packets (ping packets). For the AT-MAC algorithm, the energy

consumption of a sensor node per $\bar{T}$ is equal to the energy consumption of receiving a *sync* packet (beacon packet) plus the energy consumption of transmitting $\bar{T}/((N+1)t_m + Nt_s)$ information packets plus the energy consumption of receiving $\bar{T}/((N+1)t_m + Nt_s)$ schedule packets.

2) *Average energy consumption of the coordinator*: For the proposed algorithm, the energy consumption of the coordinator per $\bar{T}$ is equal to the energy consumption of transmitting $N+1$ *sync* packets (one for the initial synchronization and N at most for halting the network) plus the energy consumption of receiving at most N acknowledgments. As for the EH-TDMA algorithm, the energy consumption per $\bar{T}$ is equal to the energy consumption of transmitting $\bar{T}/(t_m + Nt_s)$ *sync* packets (ping packets). For the AT-MAC, the energy consumption per $\bar{T}$ is equal to the energy consumption of transmitting a *sync* packet (beacon packet) plus the energy consumption of receiving $N\bar{T}/((N+1)t_m + Nt_s)$ information packets plus the energy consumption of transmitting $\bar{T}/((N+1)t_m + Nt_s)$ schedule packets.

TABLE IV
AVERAGE ENERGY CONSUMPTION AND AVERAGE DELAY

	AT-MAC [12]	EH-TDMA [13]	Proposed
Average Energy per node (mJ)	1.95	0.95	0.076
Average Energy per coordinator (mJ)	3.85	0.95	0.196
Total Average Energy (mJ)	5.8	1.9	0.272
Average Delay (ms)	3.85	0.95	0.076

Table IV shows the average energy consumption per sensor node, the average energy consumption per coordinator (central unit), and the total average energy consumption of the proposed EH-TDMA and AT-MAC algorithms. These energies were calculated for $t_s = 56\text{ms}$ and $N = 3$, and when the *sync* packet length is $t_m = 52\mu\text{s}$ and the acknowledgment packet length is $t_a = 24\mu\text{s}$. As can be seen from the table, the proposed algorithm significantly decreases the average energy consumption, which requires less energy harvesting rate compared to EH-TDMA and AT-MAC algorithms. Compared to the EH-TDMA algorithm, the proposed algorithm reduces the average energy consumption by approximately 7 times.

3) *Average delay*: For the proposed algorithm, the time delay per $\bar{T}$ is equal to $t_m + t_a$ seconds, which is the time length of one *sync* packet plus the time length of an acknowledgment packet. As for the EH-TDMA algorithm, the time delay per $\bar{T}$ is equal to $\bar{T}t_m/(t_m + Nt_s)$ seconds, which is the total time of $\bar{T}/(t_m + Nt_s)$ *sync* packets (ping packets). For the AT-MAC algorithm, the time delay is equal to $t_m + (N+1)\bar{T}t_m/((N+1)t_m + Nt_s)$ seconds, which is the time of a *sync* packet (beacon packet) plus the time of $N\bar{T}/((N+1)t_m + Nt_s)$ information packets plus the time of $\bar{T}/((N+1)t_m + Nt_s)$ schedule packets. As can be seen from Table IV, the proposed algorithm significantly reduces the time delay compared to EH-TDMA and AT-MAC. The average delay is approximately 12 times lower than EH-TDMA. The performance of AT-MAC is the worst, which is expected because it was primarily designed to maximize channel utilization and the amount of packet transmission.

VII. CONCLUSION

We consider a single-hop wireless sensor network that consists of energy harvesting sensor nodes and a central node. The central node coordinates the network synchronization, transmissions, and data aggregation. We propose a TDMA system and an algorithm that implements this system. The proposed algorithm allows the nodes to synchronize not only based on the synchronization packets from the central node but also based on the data packets from the other nodes, which significantly reduces the average energy consumption and average delay while handling the clock drift problem between the nodes. For future work, we plan to extend the proposed TDMA system to the multi-hop scenario.

REFERENCES

- [1] D. Kandris, C. Nakas, D. Vomvas, and G. Koulouras, "Applications of wireless sensor networks: An up-to-date survey," *Applied System Innovation*, vol. 3, no. 1, 2020.
- [2] J. Lu, H. Okada, T. Itoh, T. Harada, and R. Maeda, "Toward the world smallest wireless sensor nodes with ultralow power consumption," *IEEE Sensors Journal*, vol. 14, no. 6, pp. 2035–2041, 2014.
- [3] M. S. Bandal and S. Jain, "Energy efficient protocol for wireless sensor network," in *International Conference on Recent Advances and Innovations in Engineering (ICRAIE-2014)*, 2014, pp. 1–6.
- [4] A. Razaque, M. Almiani, M. J. Khan, M. N. Kakenuly, and Y. Jararweh, "Energy efficient medium access control protocol for wireless sensor networks," in *6th International Renewable and Sustainable Energy Conference (IRSEC)*, 2018, pp. 1–6.
- [5] A. N. Sakib, M. Drieberg, and A. A. Aziz, "Energy-efficient synchronous mac protocol based on qos and multi-priority for wireless sensor networks," in *11th IEEE Symposium on Computer Applications & Industrial Electronics (ISCAIE)*, 2021, pp. 347–352.
- [6] A. Ammar and D. Reynolds, "Energy harvesting networks: Energy versus data cooperation," in *IEEE Communications Letters*, vol. 22, no. 10, 2018, pp. 2128–2131.
- [7] A. M. A. Ammar, Y. P. Fallah, and D. Reynolds, "Throughput in an energy harvesting wireless uplink," *IEEE Sensors Journal*, vol. 18, no. 6, pp. 2617–2626, 2018.
- [8] R. Cui, Z. Qu, and S. Yin, "Energy-efficient routing protocol for energy harvesting wireless sensor network," in *15th IEEE International Conference on Communication Technology*, 2013, pp. 500–504.
- [9] Y. Luo, L. Pu, and Z. Peng, "Energy stimulated time synchronization for energy harvesting wireless networks," *IEEE Transactions on Network Science and Engineering*, vol. 9, no. 3, pp. 1880–1894, 2022.
- [10] Z. A. Eu, H.-P. Tan, and W. K. Seah, "Design and performance analysis of mac schemes for wireless sensor networks powered by ambient energy harvesting," *Ad Hoc Networks*, vol. 9, no. 3, pp. 300–323, 2011.
- [11] J. Xu, J. Kan, and Y. Zhang, "Centralized energy harvesting-based tdma protocol for terahertz nanosensor networks," *Sensors*, vol. 19, no. 20, 2019.
- [12] X. Qi, K. Wang, D. Yue, L. Shu, Y. Liu, and H. Zhao, "Adaptive tdma-based mac protocol in energy harvesting wireless body area network for mobile health," in *41st Annual Conference of the IEEE Industrial Electronics Society*, 2015, pp. 004 175–004 180.
- [13] S. Kosunalp, "Eh-tdma: A tdma-based mac protocol for energy-harvesting wireless sensor networks," *International Journal of Computer Science and Information Security*, vol. 14, pp. 325–328, 08 2016.
- [14] D. Hauweele and B. Quoitin, "Toward accurate clock drift modeling in wireless sensor networks simulation," *Computer Communications*, vol. 163, pp. 1–11, 2020.
- [15] R. Tjoa, K. Chee, P. Sivaprasad, S. Rao, and J. Lim, "Clock drift reduction for relative time slot tdma-based sensor networks," in *IEEE 15th International Symposium on Personal, Indoor and Mobile Radio Communications*, vol. 2, 2004, pp. 1042–1047 Vol.2.
- [16] Pei-Shan Yu, Jemin Lee, Tony Q. S. Quek, and Y.-W. Peter Hong, "Energy harvesting personal cells-traffic offloading and network throughput," in *IEEE International Conference on Communications (ICC)*, 2015, pp. 2184–2189.